CORE JAVASCRIPT

A Complete Textbook

From Fundamentals to Advanced Concepts

Beginner to Intermediate

10 Chapters • Concepts, Code & Practice

Chapter 1

JavaScript Basics

1.1 Concept Introduction

JavaScript is a high-level, interpreted programming language that runs in web browsers and, with the help of Node.js, on servers as well. It was created in 1995 by Brendan Eich and has since become one of the most widely used programming languages in the world. Every interactive element you see on a web page — a dropdown menu, a form validation, a live search bar — is almost certainly powered by JavaScript.

At its core, JavaScript is a language that allows you to instruct a computer to perform actions. These instructions are written in a human-readable syntax that the JavaScript engine (such as V8 in Chrome, or SpiderMonkey in Firefox) reads, compiles, and executes. Understanding what happens internally when your code runs is just as important as knowing the syntax itself.

1.2 Core Theory

How JavaScript Executes

When you write JavaScript, the engine does not execute it line-by-line in a naive fashion. Modern engines use a process called Just-In-Time (JIT) compilation. The engine first parses your code into an Abstract Syntax Tree (AST), then converts it into bytecode, and finally compiles frequently used sections into optimized machine code during runtime. This is why JavaScript, despite being a scripted language, can be very fast.

Understanding this matters because JavaScript is single-threaded. It has one Call Stack — a data structure that records what functions are being executed. If one function takes too long, it blocks everything else. This is why asynchronous programming (covered in Chapter 10) exists.

Variables and Data Storage

A variable is a named container that holds a value. In JavaScript, there are three ways to declare a variable: var, let, and const. Understanding the differences between them is critical.

var is the oldest declaration keyword. It is function-scoped, meaning it is accessible anywhere within the function it is declared in. It is also "hoisted," which means JavaScript moves the declaration to the top of its scope before executing the code. This often leads to confusing bugs, which is why var is largely avoided in modern JavaScript.

let was introduced in ES6 (2015). It is block-scoped, meaning it only exists within the nearest set of curly braces { } that contains it. It can be reassigned but not re-declared in the same scope.

const is also block-scoped. A variable declared with const cannot be reassigned after its initial value is set. Note that for objects and arrays, const does not make the content immutable — it only prevents the variable from being reassigned to a different object or array.

Data Types

JavaScript has eight data types. Seven are primitive (they hold a single value) and one is non-primitive (it holds a reference to a location in memory).

The primitive types are: Number (for all numeric values, both integers and decimals), String (a sequence of characters), Boolean (true or false), undefined (a variable that has been declared but not assigned a value), null (an explicitly empty value), BigInt (for very large integers), and Symbol (a unique identifier, used in advanced scenarios).

The non-primitive type is Object, which includes plain objects, arrays, and functions. When you assign an object to a variable, you are storing a reference (a memory address), not the actual data. This has important implications when copying or comparing objects.

Type Coercion

JavaScript is a dynamically typed language. You do not declare the type of a variable; the engine figures it out at runtime. JavaScript also performs automatic type conversion, called type coercion, when operations involve mixed types. This is one of the most misunderstood aspects of the language.

When you use the + operator with a number and a string, JavaScript converts the number to a string and concatenates them. When you use -, *, or /, JavaScript tries to convert strings to numbers. Understanding this prevents unexpected bugs.

1.3 Syntax and Structure

```
// Variable Declarations
var oldWay = "avoid using var in modern code";
let score = 0;           // can be reassigned
const PI = 3.14159;     // cannot be reassigned

// Data Types
let age = 25;            // Number
let name = "Alice";      // String
let isLoggedIn = true;   // Boolean
let address;             // undefined (declared, not assigned)
```

```
let emptySlot = null;    // null (explicitly empty)

// Checking the type of a variable
typeof age;              // "number"
typeof name;             // "string"
typeof isLoggedIn;       // "boolean"
typeof address;          // "undefined"
typeof null;             // "object" — this is a known bug in JavaScript!
```

1.4 Practical Demonstration

Example 1: Variable Declarations and Reassignment

```
let counter = 0;
counter = counter + 1;  // counter is now 1
counter += 1;           // shorthand: counter is now 2

const MAX_RETRIES = 3;
// MAX_RETRIES = 5;     // Error: Assignment to constant variable
```

In this example, counter starts at 0. We reassign it twice using different notations. The const MAX_RETRIES cannot be changed — attempting to do so throws an error. Notice that constants are conventionally written in UPPER_SNAKE_CASE to signal their immutability.

Example 2: Understanding Type Coercion

```
// Addition with string — coercion to string
console.log(5 + "3");      // "53" (number 5 becomes string "5")
console.log("5" + 3);      // "53"

// Subtraction — coercion to number
console.log("10" - 4);     // 6 (string "10" becomes number 10)

// Comparison with == (loose equality)
console.log(0 == false);   // true (coercion happens)
console.log("" == false);  // true

// Comparison with === (strict equality — no coercion)
console.log(0 === false);  // false (different types)
console.log(1 === 1);      // true

// Always prefer === to avoid bugs from coercion
```

The == operator compares values after performing type coercion. This means 0 and false are considered equal because both are "falsy" values. The === operator compares both the value and the type, making it far safer and more predictable. Always use === in your code.

Example 3: String Operations

```
let firstName = "John";
let lastName  = "Doe";

// Concatenation (old style)
let full1 = firstName + " " + lastName;  // "John Doe"

// Template literals (modern, preferred)
let full2 = `${firstName} ${lastName}`;  // "John Doe"

// String properties and methods
console.log(full2.length);          // 8
console.log(full2.toUpperCase());  // "JOHN DOE"
console.log(full2.includes("Doe")); // true
console.log(full2.slice(0, 4));    // "John"
```

Template literals use backticks (`) and allow you to embed expressions inside ${} placeholders. They are the modern and preferred way to build strings in JavaScript. They also support multi-line strings without escape characters.

Example 4: Arithmetic and Operators

```
let a = 10;
let b = 3;

console.log(a + b);   // 13
console.log(a - b);   // 7
console.log(a * b);   // 30
console.log(a / b);   // 3.3333...
console.log(a % b);   // 1  (remainder/modulo)
console.log(a ** b);  // 1000 (exponentiation: 10^3)

// Increment and Decrement
let x = 5;
x++;   // x is now 6 (post-increment)
x--;   // x is now 5 (post-decrement)
++x;   // x is now 6 (pre-increment)
```

1.5 Edge Cases & Pitfalls

⚠Warning: typeof null returns "object". This is a historical bug in JavaScript that was never fixed to avoid breaking existing code. To check if something is null, always use: value === null.

⚠Warning: NaN (Not a Number) is of type "number". It is the result of an invalid arithmetic operation like 0/0 or parseInt("hello"). Checking NaN === NaN returns false — use Number.isNaN(value) instead.

```
// The var hoisting trap
console.log(myVar); // undefined — NOT an error!
var myVar = "hello";

// With let, this would throw a ReferenceError
// console.log(myLet); // ReferenceError: Cannot access before initialization
let myLet = "hello";

// NaN quirk
let result = "abc" - 5;    // NaN
console.log(result === NaN);       // false — NaN !== NaN
console.log(Number.isNaN(result)); // true — use this instead
```

Mini Practice Problems

1. Declare three variables using var, let, and const. Assign each a different data type. Try reassigning each. What happens with const?
2. Write a line of code that uses the + operator with a number and a string. Predict the result before running it, then verify.
3. What is the output of typeof null? Why is it surprising? How would you correctly check if a variable is null?
4. Write code using a template literal to display: "My name is [name] and I am [age] years old." using actual variables.
5. What is the difference between == and ===? Give one example where they produce different results.

Chapter Summary

- JavaScript is single-threaded and uses JIT compilation for performance.
- Prefer let and const over var. Use const by default; use let when reassignment is needed.
- JavaScript has 8 data types: 7 primitives (Number, String, Boolean, null, undefined, BigInt, Symbol) and one non-primitive (Object).
- Type coercion happens automatically — use === (strict equality) to avoid bugs.
- Template literals (backticks) are the preferred way to build strings.
- typeof null returns "object" — a known bug. Use === null to check for null.
- NaN is not equal to itself — always use Number.isNaN() to check for NaN.

Chapter 2

Conditions & Decision Making

2.1 Concept Introduction

Programs rarely execute every statement sequentially without deviation. Almost all real-world logic requires decisions: if a user is logged in, show the dashboard; otherwise, show the login page. If a score exceeds 90, assign grade A; if it exceeds 80, assign grade B; and so on. Conditional statements are the mechanism JavaScript provides for making such decisions.

Conditions work by evaluating an expression to a Boolean value — true or false. Based on that result, one branch of code executes and others are skipped. Understanding which values JavaScript treats as true and which it treats as false (truthy vs falsy) is essential to writing correct conditional logic.

2.2 Core Theory

Truthy and Falsy Values

In JavaScript, every value has an inherent Boolean interpretation. When a non-Boolean value is used in a condition, JavaScript automatically converts it. The following values are falsy (they evaluate to false in a Boolean context): false, 0, -0, 0n (BigInt zero), "" (empty string), null, undefined, and NaN. Every other value, including all objects, arrays, non-empty strings, and non-zero numbers, is truthy.

> **Key Insight**
> An empty array [] and an empty object {} are both truthy! This surprises many beginners. Only a completely absent value (null, undefined) or an explicitly falsy primitive evaluates to false.

Short-Circuit Evaluation

JavaScript evaluates logical expressions from left to right and stops as soon as the result is determined. In an AND (&&) expression, if the first operand is falsy, the second is never evaluated. In an OR (||) expression, if the first operand is truthy, the second is never evaluated. This is called short-circuit evaluation and is frequently used to write concise, defensive code.

The Nullish Coalescing Operator (??)

Introduced in ES2020, the ?? operator returns its right-hand side operand only when the left-hand side is null or undefined. Unlike ||, it does not treat 0 or "" as falsy. This makes it more precise for handling default values when 0 or an empty string are valid inputs.

2.3 Syntax and Structure

```
// Basic if statement
if (condition) {
  // executes if condition is truthy
}

// if-else
if (condition) {
  // executes if condition is truthy
} else {
  // executes if condition is falsy
}

// if-else if-else chain
if (condition1) {
  // ...
} else if (condition2) {
  // ...
} else {
  // fallback
}

// Ternary operator (one-line if-else)
let result = condition ? valueIfTrue : valueIfFalse;

// Switch statement
switch (expression) {
  case value1:
    // code for value1
    break;
  case value2:
    // code for value2
    break;
  default:
    // fallback code
}
```

2.4 Practical Demonstration

Example 1: Basic Conditions

```
let temperature = 35;

if (temperature > 30) {
  console.log("It is hot outside.");
} else if (temperature > 20) {
  console.log("The weather is pleasant.");
```

```
} else if (temperature > 10) {
  console.log("It is a bit cold.");
} else {
  console.log("It is freezing!");
}
// Output: "It is hot outside."
```

The conditions are checked from top to bottom. As soon as one evaluates to true, its block executes and the rest are skipped. The else block is the final fallback if none of the conditions match.

Example 2: Truthy and Falsy in Practice

```
let username = "";      // empty string — falsy

if (username) {
  console.log("Welcome, " + username);
} else {
  console.log("Please enter a username.");
}
// Output: "Please enter a username."

// Using || for default values
let displayName = username || "Guest";
console.log(displayName);  // "Guest"

// Problem: || skips 0, which may be a valid value
let retries = 0;
let count = retries || 5;   // 5 — wrong! 0 is a valid retry count

// Fix: use ?? instead
let safeCount = retries ?? 5;  // 0 — correct!
```

The || operator is useful for default values, but it treats 0 and "" as falsy. When 0 or an empty string are valid inputs, use the ?? (nullish coalescing) operator, which only falls back when the value is null or undefined.

Example 3: Ternary Operator

```
let age = 20;
let access = age >= 18 ? "Granted" : "Denied";
console.log(access);   // "Granted"

// Equivalent if-else
let access2;
if (age >= 18) {
  access2 = "Granted";
} else {
  access2 = "Denied";
```

```
}

// Nesting ternaries (use carefully — hurts readability)
let score = 85;
let grade = score >= 90 ? "A" : score >= 80 ? "B" : score >= 70 ? "C" : "F";
```

The ternary operator is ideal for simple, single-line conditions. Nesting ternaries is technically valid but should be avoided as it rapidly becomes difficult to read. Use a full if-else chain for complex multi-branch logic.

Example 4: Switch Statement

```
let day = "Monday";

switch (day) {
  case "Monday":
  case "Tuesday":
  case "Wednesday":
  case "Thursday":
  case "Friday":
    console.log("It is a weekday.");
    break;
  case "Saturday":
  case "Sunday":
    console.log("It is the weekend!");
    break;
  default:
    console.log("Unknown day.");
}
// Output: "It is a weekday."
```

Multiple cases can share the same block by stacking them without a break in between. This is called "fall-through." The break statement is critical — without it, execution continues into the next case block even if the condition does not match, which is almost always a bug.

Example 5: Short-Circuit for Safety

```
let user = null;

// Without short-circuit — this would throw an error:
// console.log(user.name); // TypeError: Cannot read property of null

// With && short-circuit — safe access
let name = user && user.name;
console.log(name);  // null (the && stopped at user because it is falsy)

// Modern optional chaining (even cleaner)
let safeName = user?.name;
console.log(safeName);  // undefined
```

The optional chaining operator (?.) introduced in ES2020 is a clean way to safely access nested properties. It returns undefined instead of throwing an error when it encounters null or undefined in the chain.

2.5 Edge Cases & Pitfalls

⚠Warning: Forgetting break in a switch statement causes fall-through. Every case must end with break (or return if inside a function) unless fall-through is intentional.

```
// The assignment vs comparison trap
let x = 5;
if (x = 10) {   // This ASSIGNS 10 to x, then evaluates to true
  console.log("This always runs");
}
// To compare, use == or ===
if (x === 10) {
  console.log("x is 10");
}

// NaN is never equal to anything, including itself
let result = NaN;
if (result === NaN) { /* never executes */ }
if (Number.isNaN(result)) { console.log("Is NaN"); }  // correct way
```

➖Mini Practice Problems

1. Write a grading system: given a score variable, print "A" (90+), "B" (80+), "C" (70+), "D" (60+), or "F" (below 60).
2. What is the output of: console.log([] == false)? Explain why. (Hint: research type coercion with ==)
3. Write a function that takes a value and returns "valid" if the value is a non-empty string and not null, otherwise "invalid". Use short-circuit logic.
4. Using a switch statement, print the number of days in a month given its name (e.g., "January" → 31). Handle February as 28.
5. What is the difference between || and ??? Write two examples where they produce different results.

✅Chapter Summary

- Conditions evaluate an expression to truthy or falsy to decide which code branch runs.
- Falsy values: false, 0, "", null, undefined, NaN. Everything else is truthy.
- Use === for comparisons; == performs type coercion and leads to unexpected results.
- The ternary operator is useful for simple inline conditions but avoid nesting it deeply.

- switch is best for checking one value against many possible cases — always include break.
- || provides defaults but skips 0 and "". Use ?? to only skip null and undefined.
- Optional chaining (?.) safely accesses nested properties without throwing errors.

Chapter 3

Loops & Iteration

3.1 Concept Introduction

A loop is a control structure that repeats a block of code multiple times, as long as a specified condition remains true. Without loops, you would have to write the same code repeatedly for repetitive tasks. With loops, you can process a list of 1,000 items with the same code you would use for a list of 1.

JavaScript provides several looping constructs: for, while, do...while, for...of, and for...in. Each has its own use case. Choosing the right loop improves both code clarity and, in some cases, performance.

3.2 Core Theory

How Loops Work Internally

Every loop has three key components: initialization (where do we start?), a condition (when do we stop?), and an update (how do we progress?). A loop that never reaches a stopping condition runs forever — this is called an infinite loop, and it crashes the browser or Node.js process.

The for loop executes a fixed number of times determined by a counter variable. The while loop is more flexible — it repeats as long as any condition is true, making it suitable when the number of iterations is not known in advance.

break and continue

The break statement immediately exits the loop, skipping any remaining iterations. The continue statement skips the rest of the current iteration and moves directly to the next one. Both are useful tools for controlling loop flow, but overuse can make code harder to follow.

3.3 Syntax and Structure

```
// for loop
for (initialization; condition; update) {
  // loop body
}

// while loop
while (condition) {
```

```
  // loop body
  // must include an update to avoid infinite loop
}

// do...while loop (executes body at least once)
do {
  // loop body
} while (condition);

// for...of (iterates over iterable values: arrays, strings)
for (let item of iterable) {
  // use item
}

// for...in (iterates over object keys)
for (let key in object) {
  // use key
}
```

3.4 Practical Demonstration

Example 1: The for Loop

```
// Print numbers 1 to 5
for (let i = 1; i <= 5; i++) {
  console.log(i);
}
// Output: 1 2 3 4 5 (each on new line)

// Count backwards
for (let i = 5; i >= 1; i--) {
  console.log(i);
}
// Output: 5 4 3 2 1

// Loop with step size of 2
for (let i = 0; i <= 10; i += 2) {
  console.log(i);
}
// Output: 0 2 4 6 8 10
```

The for loop has three parts separated by semicolons inside the parentheses: let i = 1 initializes the counter, i <= 5 is the condition checked before each iteration, and i++ updates the counter after each iteration. When i becomes 6, the condition is false and the loop stops.

Example 2: while Loop

```
// Simulate waiting for a condition
let attempts = 0;
let success = false;

while (!success && attempts < 5) {
  attempts++;
  console.log("Attempt " + attempts);

  if (attempts === 3) {
    success = true;
    console.log("Connected!");
  }
}
// Output: Attempt 1, Attempt 2, Attempt 3, Connected!
```

The while loop is ideal here because we do not know in advance at which attempt we will succeed. The loop continues as long as success is false and attempts is below 5. If success is never achieved, the attempts limit prevents an infinite loop.

Example 3: for...of Loop

```
let fruits = ["apple", "banana", "cherry"];

// Old way using index
for (let i = 0; i < fruits.length; i++) {
  console.log(fruits[i]);
}

// Modern way using for...of
for (let fruit of fruits) {
  console.log(fruit);
}
// Output: apple, banana, cherry

// Works with strings too
for (let char of "hello") {
  console.log(char);
}
// Output: h, e, l, l, o
```

Example 4: break and continue

```
// break — stop when we find what we want
let numbers = [3, 7, 12, 5, 9, 14];

for (let num of numbers) {
  if (num > 10) {
    console.log("First number above 10:", num);
    break;  // stop searching
```

```
  }
}
// Output: "First number above 10: 12"

// continue — skip specific iterations
for (let i = 1; i <= 10; i++) {
  if (i % 2 === 0) continue;  // skip even numbers
  console.log(i);
}
// Output: 1 3 5 7 9
```

Example 5: Nested Loops (Multiplication Table)

```
// A nested loop — the inner loop runs completely for each outer iteration
for (let i = 1; i <= 3; i++) {
  for (let j = 1; j <= 3; j++) {
    console.log(`${i} x ${j} = ${i * j}`);
  }
}
// Output:
// 1 x 1 = 1
// 1 x 2 = 2
// 1 x 3 = 3
// 2 x 1 = 2
// ... and so on
```

Nested loops are powerful for working with grids, matrices, and combinations, but be aware of performance. A double-nested loop with n iterations each runs n^2 times. Triple nesting runs n^3 times. Keep nesting shallow where possible.

3.5 Edge Cases & Pitfalls

⚠Warning: Infinite loops crash the browser or process. Always ensure the loop condition will eventually become false. Common mistake: using = instead of += in the update step, so the counter never changes.

```
// Off-by-one error — very common
let arr = [1, 2, 3];

// WRONG: i <= arr.length accesses index 3 which is undefined
for (let i = 0; i <= arr.length; i++) {
  console.log(arr[i]);  // prints 1, 2, 3, then undefined
}

// CORRECT: i < arr.length
```

```
for (let i = 0; i < arr.length; i++) {
  console.log(arr[i]);  // prints 1, 2, 3
}

// for...in on arrays — avoid this
let arr2 = [10, 20, 30];
for (let key in arr2) {
  console.log(key);   // "0", "1", "2" — gives INDEX, not value
}
// Use for...of for arrays, for...in only for plain objects
```

Mini Practice Problems

1. Write a loop that calculates the sum of all numbers from 1 to 100.
2. Using a for loop, find and print all multiples of 7 between 1 and 100.
3. Write a while loop that keeps dividing a number by 2 until it is less than 1. Count how many divisions were needed.
4. Print the following pattern using nested loops: * ** *** ****
5. Write a loop that iterates through an array of numbers and creates a new array containing only the even numbers. Do not use any built-in array methods.

Chapter Summary

- A loop repeats a block of code while a condition is true.
- for loops are best when the number of iterations is known. while loops are best when it is not.
- do...while guarantees the body executes at least once.
- for...of iterates over values of iterables (arrays, strings). for...in iterates over keys of objects.
- break exits the loop immediately. continue skips to the next iteration.
- Off-by-one errors (using <= instead of <) are the most common loop bug.
- Avoid using for...in on arrays — use for...of or a standard for loop instead.

Chapter 4

Functions

4.1 Concept Introduction

A function is a named, reusable block of code designed to perform a specific task. Functions are the primary mechanism for organizing code, avoiding repetition, and building abstractions. Instead of writing the same logic multiple times, you define it once in a function and call it whenever needed.

Functions are first-class citizens in JavaScript. This means they can be stored in variables, passed as arguments to other functions, and returned from other functions — just like any other value. This property is what enables powerful patterns like higher-order functions, callbacks, and closures.

4.2 Core Theory

Function Declaration vs Function Expression

There are two primary ways to define a function. A function declaration uses the function keyword followed by a name and is hoisted to the top of its scope, meaning you can call it before it appears in the code. A function expression assigns a function (which may be anonymous) to a variable and is not hoisted — you cannot call it before the line where it is defined.

The Call Stack

When a function is called, JavaScript pushes a new execution context onto the call stack. This context contains the function's local variables, its arguments, and a reference to the outer scope. When the function returns, its context is popped off the stack and control returns to the caller. If functions call functions call functions, the stack grows. If it grows too large (due to uncontrolled recursion), a "Maximum call stack size exceeded" error occurs — commonly called a stack overflow.

Parameters, Arguments, and Return Values

Parameters are the variables listed in the function definition. Arguments are the actual values passed when calling the function. If you call a function with fewer arguments than parameters, the missing parameters are undefined. If you pass more arguments than parameters, the extras are accessible via the arguments object (in regular functions) or simply ignored.

A function returns a value using the return keyword. Execution stops immediately at return. If no return is specified, the function returns undefined implicitly.

Arrow Functions

Arrow functions (introduced in ES6) provide a shorter syntax for function expressions. They are ideal for simple, single-expression functions. There is, however, a critical difference: arrow functions do not have their own this binding. They inherit this from the surrounding context. This makes them unsuitable for object methods (where you need to refer to the object itself) but ideal for callbacks.

4.3 Syntax and Structure

```
// Function Declaration (hoisted)
function greet(name) {
  return "Hello, " + name;
}

// Function Expression (not hoisted)
const add = function(a, b) {
  return a + b;
};

// Arrow Function
const multiply = (a, b) => a * b;

// Arrow with a body block (for multiple statements)
const clamp = (value, min, max) => {
  if (value < min) return min;
  if (value > max) return max;
  return value;
};

// Default Parameters
function power(base, exponent = 2) {
  return base ** exponent;
}
power(5);     // 25 (exponent defaults to 2)
power(5, 3);  // 125

// Rest Parameters (collects remaining args into an array)
function sum(...numbers) {
  return numbers.reduce((total, n) => total + n, 0);
}
sum(1, 2, 3, 4, 5);  // 15
```

4.4 Practical Demonstration

Example 1: Basic Function

```
function calculateArea(width, height) {
  const area = width * height;
  return area;
}

let roomArea = calculateArea(12, 8);
console.log(roomArea);  // 96

// The function can be reused with different arguments
let officeArea = calculateArea(20, 15);
console.log(officeArea);  // 300
```

Example 2: Hoisting Difference

```
// Function declaration — can be called before definition
console.log(square(4));  // 16 — works due to hoisting

function square(n) {
  return n * n;
}

// Function expression — cannot be called before definition
// console.log(cube(3));  // Error: Cannot access before initialization

const cube = function(n) {
  return n * n * n;
};

console.log(cube(3));  // 27
```

Example 3: Arrow Functions and this

```
const counter = {
  count: 0,

  // Regular function — has its own "this" (refers to counter object)
  increment: function() {
    this.count++;
    console.log(this.count);
  },

  // Arrow function — no own "this", inherits from surrounding scope
  // In this context, "this" would be the outer scope (Window/global)
  // NOT the counter object — so avoid arrows for methods
  badIncrement: () => {
```

```
    // this.count would be undefined here in strict mode
  }
};

counter.increment();  // 1
counter.increment();  // 2

// Arrow functions shine as callbacks
let numbers = [1, 2, 3, 4, 5];
let doubled = numbers.map(n => n * 2);
console.log(doubled);  // [2, 4, 6, 8, 10]
```

Example 4: Recursion

```
// A function that calls itself is recursive
// Base case: prevents infinite recursion
// Recursive case: the function calls itself with a smaller input

function factorial(n) {
  if (n <= 1) return 1;         // base case
  return n * factorial(n - 1);  // recursive case
}

console.log(factorial(5));
// 5 * factorial(4)
// 5 * 4 * factorial(3)
// 5 * 4 * 3 * factorial(2)
// 5 * 4 * 3 * 2 * factorial(1)
// 5 * 4 * 3 * 2 * 1 = 120
```

Recursion is a function calling itself with a progressively simpler version of the problem, until it reaches a base case that requires no further recursion. Every recursive function must have a base case — without it, the function would call itself forever, causing a stack overflow.

Example 5: Functions as Values (First-Class)

```
// Storing a function in a variable
const greet = function(name) { return `Hello, ${name}!`; };

// Passing a function as an argument
function applyToFive(fn) {
  return fn(5);
}

function double(n) { return n * 2; }
function triple(n) { return n * 3; }

console.log(applyToFive(double)); // 10
console.log(applyToFive(triple)); // 15
```

```
// Returning a function from another function
function multiplier(factor) {
  return (number) => number * factor;
}

const double2 = multiplier(2);
const triple2 = multiplier(3);
console.log(double2(7));  // 14
console.log(triple2(7));  // 21
```

4.5 Edge Cases & Pitfalls

⚠Warning: If a function has no explicit return statement, or the return is reached with no value, it returns undefined. This commonly causes bugs when the caller expects a number or string.

```
// Arguments object vs rest parameters
function oldStyle() {
  console.log(arguments);  // array-like object (not a real array)
  // arguments.forEach(...) would fail!
}

// Modern approach — rest parameters give a real array
function newStyle(...args) {
  args.forEach(arg => console.log(arg));  // works fine
}

// Arrow functions do NOT have an arguments object
const arrowFn = (...args) => {
  console.log(args);  // use rest parameter instead
};
```

➖Mini Practice Problems

1. Write a recursive function that computes the nth Fibonacci number. (F(0) = 0, F(1) = 1, F(n) = F(n-1) + F(n-2))
2. Write a function isPalindrome(str) that returns true if the string reads the same forwards and backwards.
3. Write a function makeCounter() that returns a new function. Each call to that returned function should increment and return a count starting from 0.
4. What is the difference between a function declaration and a function expression in terms of hoisting? Give an example.
5. Write a function that accepts any number of arguments and returns their average using rest parameters.

✓Chapter Summary

- Functions are reusable blocks of code that accept inputs (parameters) and return outputs.
- Function declarations are hoisted; function expressions are not.
- Arrow functions have shorter syntax and do not have their own this — use them for callbacks, not object methods.
- Default parameters provide fallback values. Rest parameters collect extra arguments into an array.
- Functions are first-class values — they can be stored, passed, and returned like any other value.
- Recursion is a function calling itself; always define a base case to prevent infinite recursion.
- If no return statement exists, a function implicitly returns undefined.

Chapter 5

Arrays

5.1 Concept Introduction

An array is an ordered collection of values stored in a single variable. Arrays allow you to group related data together and process it systematically using loops and methods. In JavaScript, arrays are dynamic — they can grow or shrink at runtime, and they can hold values of any type, including other arrays (creating multi-dimensional arrays).

Arrays are technically objects in JavaScript. Each element is stored at a numeric index starting at 0. The last valid index in an array of length n is n - 1. Accessing an index outside this range returns undefined, not an error.

5.2 Core Theory

Array Internal Structure

In JavaScript, arrays are objects where the keys are numeric strings ("0", "1", "2", etc.) and there is a special length property. When you write arr[0], JavaScript converts 0 to the string "0" and looks up that property. This is why arrays in JavaScript are more flexible than arrays in lower-level languages — but also why they can behave unexpectedly if you use non-numeric keys.

Mutating vs Non-Mutating Methods

This is one of the most important distinctions in array programming. Some methods modify (mutate) the original array in place: push, pop, shift, unshift, splice, reverse, sort. Other methods return a new array and leave the original unchanged: map, filter, reduce, slice, concat, flat, flatMap. Knowing which is which prevents subtle bugs, especially when multiple parts of your code share a reference to the same array.

Reference vs Value

When you assign an array to a new variable, you are copying the reference, not the array itself. Both variables point to the same array in memory. Modifying one changes the other. To create an independent copy, use the spread operator [...arr], Array.from(arr), or arr.slice().

5.3 Syntax and Structure

```
// Array creation
```

```
let empty = [];
let fruits = ["apple", "banana", "cherry"];
let mixed = [1, "hello", true, null, { id: 1 }];

// Accessing elements
fruits[0];          // "apple" (first element)
fruits[fruits.length - 1]; // "cherry" (last element)

// Adding and removing
fruits.push("date");     // adds to end → length becomes 4
fruits.pop();            // removes from end → returns "date"
fruits.unshift("avocado"); // adds to beginning
fruits.shift();          // removes from beginning

// splice(start, deleteCount, ...itemsToInsert)
fruits.splice(1, 1, "blueberry"); // removes 1 at index 1, inserts
"blueberry"
```

5.4 Practical Demonstration

Example 1: Array Iteration

```
let scores = [85, 92, 78, 95, 88];

// forEach — iterates but does not return a new array
scores.forEach((score, index) => {
  console.log(`Score ${index + 1}: ${score}`);
});

// Find the total manually
let total = 0;
for (let score of scores) {
  total += score;
}
let average = total / scores.length;
console.log("Average:", average);  // 87.6
```

Example 2: map, filter, reduce

```
let numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10];

// map — transform each element (returns new array)
let squared = numbers.map(n => n * n);
console.log(squared);  // [1, 4, 9, 16, 25, 36, 49, 64, 81, 100]

// filter — keep elements that pass a test (returns new array)
let evens = numbers.filter(n => n % 2 === 0);
```

```
console.log(evens);    // [2, 4, 6, 8, 10]

// reduce — accumulate to a single value
let sum = numbers.reduce((accumulator, current) => accumulator + current, 0);
console.log(sum);      // 55

// Chain them: sum of squares of even numbers
let result = numbers
  .filter(n => n % 2 === 0)
  .map(n => n * n)
  .reduce((acc, n) => acc + n, 0);
console.log(result);   // 4 + 16 + 36 + 64 + 100 = 220
```

Chaining array methods is one of the most powerful patterns in JavaScript. Notice how the code reads almost like an English sentence: filter the evens, square them, then sum the results. Each method returns an array (except reduce), which is immediately fed into the next method.

Example 3: Searching Arrays

```
let products = ["laptop", "phone", "tablet", "headphones"];

// includes — does the array contain this value?
console.log(products.includes("phone"));    // true
console.log(products.includes("charger"));  // false

// indexOf — what position is this value at?
console.log(products.indexOf("tablet"));    // 2
console.log(products.indexOf("charger"));   // -1 (not found)

// find — first element that matches a condition
let longName = products.find(p => p.length > 6);
console.log(longName);  // "headphones" (first match)

// findIndex — index of first match
let idx = products.findIndex(p => p.length > 6);
console.log(idx);       // 3

// some — does any element pass the test?
console.log(products.some(p => p.startsWith("t")));  // true (tablet)

// every — do ALL elements pass the test?
console.log(products.every(p => p.length > 3));      // true
```

Example 4: Copying and Spreading

```
let original = [1, 2, 3];

// WRONG: this copies the reference, not the array
let wrongCopy = original;
```

```
wrongCopy.push(4);
console.log(original);   // [1, 2, 3, 4] — original was modified!

// CORRECT: shallow copy with spread operator
let correctCopy = [...original];
correctCopy.push(5);
console.log(original);   // [1, 2, 3, 4] — unchanged

// Merging arrays
let a = [1, 2, 3];
let b = [4, 5, 6];
let merged = [...a, ...b];
console.log(merged);  // [1, 2, 3, 4, 5, 6]
```

5.5 Edge Cases & Pitfalls

⚠Warning: sort() converts elements to strings by default and sorts alphabetically. This means [1, 10, 2].sort() gives [1, 10, 2] sorted as strings: [1, 10, 2]. For numeric sorting, always provide a comparator: arr.sort((a, b) => a - b).

```
// The sort trap
let nums = [10, 1, 21, 2];
console.log(nums.sort());             // [1, 10, 2, 21] — WRONG
console.log(nums.sort((a,b) => a-b)); // [1, 2, 10, 21] — CORRECT

// Sparse arrays
let sparse = [1, , , 4];   // holes at index 1 and 2
console.log(sparse.length);  // 4
console.log(sparse[1]);      // undefined
// Some methods skip holes, others include them — avoid sparse arrays
```

➖Mini Practice Problems

1. Given an array of student objects [{name, grade}], use filter to get students who passed (grade >= 60), and map to return just their names.
2. Write a function that removes all duplicate values from an array and returns a new array. (Hint: use a Set or filter with indexOf)
3. Flatten the array [[1, 2], [3, 4], [5, 6]] into [1, 2, 3, 4, 5, 6] without using the flat() method.
4. Write a function that rotates an array by k positions to the left. For example: rotate([1,2,3,4,5], 2) → [3,4,5,1,2].
5. Explain what happens when you do: const a = [1,2,3]; const b = a; b.push(4); console.log(a). Why does this occur and how do you prevent it?

✓Chapter Summary

- Arrays are ordered, zero-indexed collections. Accessing an out-of-bounds index returns undefined.
- Key mutating methods: push, pop, shift, unshift, splice, sort, reverse.
- Key non-mutating methods: map, filter, reduce, slice, find, includes, concat.
- Assigning an array to another variable copies the reference, not the data. Use spread [...arr] to copy.
- sort() is alphabetical by default — always use a comparator for numbers.
- Chain map, filter, and reduce to write expressive, readable data transformations.
- forEach is for side effects only — it returns undefined, not a new array.

Chapter 6

Objects

6.1 Concept Introduction

An object is a collection of key-value pairs, where each key is a string (or Symbol) and each value can be any data type, including another object or a function. Objects are the fundamental building block of JavaScript and almost everything in the language is, at some level, an object — including arrays, functions, and even regular expressions.

Objects are used to model real-world entities. A user can be an object with properties like name, email, and age. A product can be an object with properties like id, title, price, and a method called applyDiscount(). Understanding objects deeply is prerequisite to understanding classes, prototypes, and the entire ecosystem of JavaScript patterns.

6.2 Core Theory

Property Access

Object properties can be accessed using dot notation (obj.property) or bracket notation (obj["property"]). Dot notation is cleaner and more common, but bracket notation is required when the property name is stored in a variable or when the name contains special characters or spaces.

Prototypes and Prototype Chain

Every object in JavaScript has an internal link to another object called its prototype. When you try to access a property on an object and it is not found, JavaScript automatically looks up the prototype chain — checking the prototype, then the prototype's prototype, and so on, until it either finds the property or reaches the end of the chain (null). This is JavaScript's inheritance mechanism.

When you create a plain object with {}, its prototype is Object.prototype, which provides methods like toString(), hasOwnProperty(), and valueOf(). This is why those methods are available on every object even though you never defined them.

this Keyword in Objects

Inside an object method, the this keyword refers to the object that the method was called on. This is called the execution context. The value of this is not determined by where the function is defined — it is determined by how the function is called. This dynamic nature of this is one of the most nuanced aspects of JavaScript.

6.3 Syntax and Structure

```
// Object literal syntax
const person = {
  name: "Alice",
  age: 30,
  isEmployed: true,
  address: {            // nested object
    city: "New York",
    zip: "10001"
  },
  greet() {             // shorthand method syntax (ES6)
    return `Hi, I am ${this.name}`;
  }
};

// Property access
person.name;            // "Alice" (dot notation)
person["age"];          // 30 (bracket notation)

// Dynamic property access
let key = "isEmployed";
person[key];            // true

// Adding and deleting properties
person.email = "alice@example.com";  // add new property
delete person.isEmployed;            // remove a property
```

6.4 Practical Demonstration

Example 1: Object Methods and this

```
const bankAccount = {
  owner: "Bob",
  balance: 1000,

  deposit(amount) {
    this.balance += amount;
    return this;  // enables method chaining
  },

  withdraw(amount) {
    if (amount > this.balance) {
      console.log("Insufficient funds");
      return this;
    }
    this.balance -= amount;
    return this;
```

```
  },

  getBalance() {
    return `${this.owner}'s balance: $${this.balance}`;
  }
};

// Method chaining
bankAccount.deposit(500).withdraw(200).getBalance();
// "Bob's balance: $1300"
```

Example 2: Destructuring

```
const user = {
  id: 101,
  name: "Carol",
  email: "carol@example.com",
  role: "admin"
};

// Extract specific properties into variables
const { name, role } = user;
console.log(name);  // "Carol"
console.log(role);  // "admin"

// Rename during destructuring
const { name: userName, email: userEmail } = user;
console.log(userName);   // "Carol"

// Default values in destructuring
const { name: n, age = 25 } = user;
console.log(n);    // "Carol"
console.log(age);  // 25 (user.age was undefined, so default applies)

// Nested destructuring
const { address: { city } = {} } = user;
```

Example 3: Spread and Object Copying

```
const defaults = { theme: "light", fontSize: 14, language: "en" };
const userPrefs = { theme: "dark", fontSize: 16 };

// Merge objects — later properties overwrite earlier ones
const config = { ...defaults, ...userPrefs };
console.log(config);
// { theme: "dark", fontSize: 16, language: "en" }

// Shallow copy of an object
const original = { a: 1, b: { c: 2 } };
```

```
const copy = { ...original };

copy.a = 99;
console.log(original.a);  // 1 — primitive, not affected

copy.b.c = 99;
console.log(original.b.c);  // 99 — nested object is still shared!
// Spread creates a SHALLOW copy only
```

The spread operator creates a shallow copy, meaning only the top-level properties are copied. Nested objects are still shared by reference. For a deep copy, use structuredClone(obj) (modern JavaScript) or JSON.parse(JSON.stringify(obj)) for simple data.

Example 4: Iterating Over Objects

```
const scores = { Alice: 95, Bob: 82, Carol: 78, David: 91 };

// for...in loop (gives keys)
for (let name in scores) {
  console.log(`${name}: ${scores[name]}`);
}

// Object.keys() — array of keys
Object.keys(scores);    // ["Alice", "Bob", "Carol", "David"]

// Object.values() — array of values
Object.values(scores);  // [95, 82, 78, 91]

// Object.entries() — array of [key, value] pairs
Object.entries(scores).forEach(([name, score]) => {
  console.log(`${name} scored ${score}`);
});

// Find highest scorer
let topScorer = Object.entries(scores)
  .reduce((best, [name, score]) => score > best[1] ? [name, score] : best);
console.log(topScorer);  // ["Alice", 95]
```

6.5 Edge Cases & Pitfalls

⚠Warning: Checking if a property exists: do not use obj.prop === undefined because the property may exist but explicitly be set to undefined. Use "prop" in obj to check for existence, or obj.hasOwnProperty("prop").

```
// Comparing objects
```

```
const a = { x: 1 };
const b = { x: 1 };
console.log(a === b);  // false — different references in memory

// Two variables pointing to the same object
const c = a;
console.log(a === c);  // true — same reference

// The "this" context trap
const obj = {
  name: "Test",
  getName: function() { return this.name; }
};

const fn = obj.getName;
// fn();  // undefined in strict mode — "this" is not obj when called this
way
// Always call the method on the object: obj.getName()
```

Mini Practice Problems

1. Create an object representing a book with properties: title, author, year, and pages. Add a method summary() that returns a formatted string describing the book.
2. Write a function that accepts two objects and merges them, with the second object's properties taking precedence. Use the spread operator.
3. Given an array of product objects [{name, price, quantity}], compute the total inventory value (sum of price * quantity for each product).
4. What is the prototype chain? What happens when you call toString() on a plain object {} and why does it work?
5. Write a function deepEqual(a, b) that returns true if two objects have identical structure and values (handle nested objects).

Chapter Summary

- Objects are key-value stores. Keys are strings (or Symbols); values can be anything.
- Use dot notation for clean access; use bracket notation for dynamic keys or special characters.
- Every object has a prototype. Property lookup travels up the prototype chain until found or null is reached.
- this inside a method refers to the calling object — its value depends on how the function is called, not where it is defined.
- Destructuring extracts properties into variables cleanly. Spread merges or copies objects (shallow only).
- Object.keys(), Object.values(), and Object.entries() convert object data into arrays for easier processing.
- Objects are compared by reference, not by value. Two separate objects with the same content are not equal (===).

Chapter 7

Scope & Closures

7.1 Concept Introduction

Scope defines the accessibility of variables — where in your code a particular variable can be read or written. Understanding scope is essential for writing predictable code and avoiding bugs caused by unintended variable access or modification.

A closure is one of JavaScript's most powerful and often misunderstood features. It occurs when a function "remembers" the variables from its outer scope even after that outer scope has finished executing. Closures are what make patterns like module systems, data encapsulation, and callbacks work correctly.

7.2 Core Theory

Types of Scope

Global scope: Variables declared outside any function or block are global and accessible anywhere in the program. Excessive use of global variables leads to naming conflicts and unpredictable behavior.

Function scope: Variables declared with var inside a function are accessible only within that function. let and const work similarly but are additionally constrained to their block.

Block scope: Variables declared with let or const inside a block { } (if statements, loops, etc.) exist only within that block. This is a critical improvement over var, which ignores block boundaries.

Lexical scope: When a function is defined, it captures a reference to the scope in which it was written, not the scope in which it is called. This is also called static scope. The inner function can always access variables from its outer functions.

How Closures Work Internally

When a function is created in JavaScript, it stores a reference to its surrounding scope in an internal property called [[Environment]] (or the "scope chain"). When the function is later called — even if the outer function has already returned — the inner function still has access to the outer function's variables through this stored reference.

The outer function's variables are kept alive in memory as long as the inner function exists. This is how closures "close over" variables. This is not a memory leak — it is intentional design, but it is worth being aware of for long-lived closures that capture large data structures.

The Scope Chain

When JavaScript resolves a variable name, it starts looking in the current scope. If not found, it moves to the outer scope, then that scope's outer scope, and so on, until it reaches the global scope. If the variable is not found there either, it throws a ReferenceError. This outward traversal is the scope chain.

7.3 Syntax and Structure

```
// Scope hierarchy
let globalVar = "I am global";

function outer() {
  let outerVar = "I am in outer";

  function inner() {
    let innerVar = "I am in inner";
    console.log(innerVar);   // accessible — own scope
    console.log(outerVar);   // accessible — outer scope (closure)
    console.log(globalVar);  // accessible — global scope
  }

  inner();
  // console.log(innerVar);  // ReferenceError — out of scope
}

outer();
// console.log(outerVar);  // ReferenceError — out of scope
```

7.4 Practical Demonstration

Example 1: Basic Closure

```
function makeCounter() {
  let count = 0;       // this variable is "closed over"

  return function() {  // this function is the closure
    count++;
    return count;
  };
}

const counter1 = makeCounter();
const counter2 = makeCounter();

console.log(counter1());  // 1
```

```
console.log(counter1());  // 2
console.log(counter1());  // 3

console.log(counter2());  // 1 — independent closure!
console.log(counter2());  // 2
```

When makeCounter() is called, it creates a local variable count and returns an inner function. Even though makeCounter() has finished executing, the inner function still has access to count through the closure. Each call to makeCounter() creates a separate closure with its own count — counter1 and counter2 are completely independent.

Example 2: Data Privacy with Closures

```
// The Module Pattern — using closures to create private state
function createWallet(initialBalance) {
  let balance = initialBalance;  // private — not accessible from outside

  return {
    deposit(amount) {
      if (amount > 0) balance += amount;
    },
    withdraw(amount) {
      if (amount > 0 && amount <= balance) balance -= amount;
      else console.log("Invalid withdrawal");
    },
    getBalance() {
      return balance;  // returns value, but does not expose the variable
    }
  };
}

const wallet = createWallet(100);
wallet.deposit(50);
wallet.withdraw(30);
console.log(wallet.getBalance());  // 120
// console.log(wallet.balance);    // undefined — balance is private!
```

This is the Module Pattern — one of the most important design patterns in JavaScript. By returning an object of methods that close over a variable, you create true private state. External code can interact with balance only through the controlled interface you expose.

Example 3: The Classic Loop Closure Problem

```
// PROBLEM: var is function-scoped, so all closures share the same i
for (var i = 0; i < 3; i++) {
  setTimeout(function() {
    console.log(i);  // prints 3, 3, 3 — not 0, 1, 2!
  }, 1000);
```

```
}

// SOLUTION 1: Use let (block-scoped — each iteration has its own i)
for (let i = 0; i < 3; i++) {
  setTimeout(function() {
    console.log(i);  // prints 0, 1, 2 — correct!
  }, 1000);
}

// SOLUTION 2: Use an IIFE to capture the value (pre-ES6 solution)
for (var i = 0; i < 3; i++) {
  (function(j) {
    setTimeout(function() {
      console.log(j);  // 0, 1, 2
    }, 1000);
  })(i);
}
```

This is one of the most famous JavaScript interview questions. With var, there is only one i variable shared across all iterations. By the time the setTimeout callbacks run (1 second later), the loop has finished and i is 3. Using let creates a new i for each iteration, so each closure captures a different value.

Example 4: Memoization (Practical Closure Use)

```
// Memoize: cache expensive function results using a closure
function memoize(fn) {
  const cache = {};  // this stays alive due to closure

  return function(...args) {
    const key = JSON.stringify(args);
    if (cache[key] !== undefined) {
      console.log("From cache");
      return cache[key];
    }
    const result = fn(...args);
    cache[key] = result;
    return result;
  };
}

function expensiveCalc(n) {
  // Simulate heavy computation
  return n * n;
}

const memoized = memoize(expensiveCalc);
memoized(10);   // computed: 100
memoized(10);   // From cache: 100
memoized(20);   // computed: 400
```

7.5 Edge Cases & Pitfalls

⚠Warning: Variables declared without let, const, or var inside a function are automatically placed in global scope (in non-strict mode). Always declare variables properly, and use "use strict" to catch this mistake.

```
// Variable shadowing
let value = "global";

function test() {
  let value = "local";  // shadows the global "value"
  console.log(value);   // "local"
}

test();
console.log(value);   // "global" — unchanged

// Temporal Dead Zone (TDZ)
// let and const are hoisted but NOT initialized
// Accessing them before their declaration throws a ReferenceError
{
  // console.log(x);  // ReferenceError: Cannot access "x" before init
  let x = 10;
  console.log(x);    // 10
}
```

Mini Practice Problems

1. Write a function makeAdder(x) that returns a new function. The returned function should accept a number y and return x + y. Call makeAdder(5)(3) and verify it returns 8.
2. Explain the output of the classic loop closure problem with var. Why does it print 3, 3, 3 instead of 0, 1, 2?
3. Create a function createStack() that uses closure to implement a private array. Return push, pop, and peek methods.
4. What is the Temporal Dead Zone? Write a code example that demonstrates it.
5. Implement a once(fn) function that uses closure to ensure the passed function can only ever be called once, regardless of how many times once(fn) is invoked.

Chapter Summary

- Scope determines where variables are accessible. JavaScript uses lexical (static) scope.
- Block scope (let, const) is more predictable than function scope (var). Prefer let and const.
- A closure is created when an inner function retains access to its outer scope even after the outer function has returned.

- Closures enable data privacy (the Module Pattern), stateful functions, and memoization.
- The classic loop closure bug with var occurs because all iterations share the same variable. Fix it with let.
- The Temporal Dead Zone applies to let and const — they are hoisted but not initialized, so accessing them before declaration throws an error.
- Closures hold references to variables, not copies. Changes to the variable are reflected in the closure.

Chapter 8

Higher-Order Functions

8.1 Concept Introduction

A higher-order function (HOF) is a function that either accepts another function as an argument, returns a function, or both. This concept builds directly on the idea that functions are first-class values in JavaScript.

Higher-order functions are the foundation of functional programming in JavaScript. They allow you to write more abstract, reusable, and composable code. Instead of writing explicit loops and conditionals for every task, you describe what you want to do — and let the HOF handle how to do it.

8.2 Core Theory

Callbacks

A callback is a function passed as an argument to another function, to be called at a later point. You have already used callbacks with array methods like map, filter, and forEach. The array method (the HOF) handles the iteration; your callback defines what to do on each element.

Function Composition

Function composition is the practice of combining multiple functions to build a more complex one. The output of one function becomes the input of the next. This creates a pipeline of transformations that is easy to reason about, test, and extend.

Currying

Currying is the transformation of a function that takes multiple arguments into a sequence of functions that each take one argument. A curried function returns a new function for each argument until all arguments have been received, at which point it computes and returns the final result. Currying enables partial application — creating specialized functions from general ones.

8.3 Syntax and Structure

```
// Higher-order function that accepts a function
function repeat(n, fn) {
  for (let i = 0; i < n; i++) fn(i);
}
```

```
repeat(3, i => console.log(`Step ${i + 1}`));
// Step 1, Step 2, Step 3

// Higher-order function that returns a function
function multiplier(factor) {
  return n => n * factor;
}

const double = multiplier(2);
const triple = multiplier(3);

// Curried function
const add = a => b => a + b;
const add5 = add(5);
console.log(add5(3));  // 8
console.log(add5(7));  // 12
```

8.4 Practical Demonstration

Example 1: Custom HOFs

```
// Re-implementing filter from scratch
function myFilter(array, predicate) {
  const result = [];
  for (let item of array) {
    if (predicate(item)) result.push(item);
  }
  return result;
}

const nums = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10];
const evens = myFilter(nums, n => n % 2 === 0);
console.log(evens);  // [2, 4, 6, 8, 10]

// The "predicate" parameter is interchangeable
const above5 = myFilter(nums, n => n > 5);
console.log(above5);  // [6, 7, 8, 9, 10]
```

By accepting a predicate (a function that returns true or false), myFilter becomes completely flexible. The caller decides what the filtering criterion is — the HOF only provides the iteration mechanism. This separation of concerns is the essence of functional programming.

Example 2: Function Composition

```
const trim = str => str.trim();
```

```
const toLower = str => str.toLowerCase();
const removeSpaces = str => str.replace(/\s+/g, "-");

// Manual composition (right to left)
const toSlug = str => removeSpaces(toLower(trim(str)));
console.log(toSlug("  Hello World  "));  // "hello-world"

// Generic compose function (right to left)
function compose(...fns) {
  return function(value) {
    return fns.reduceRight((acc, fn) => fn(acc), value);
  };
}

// Generic pipe function (left to right) — more intuitive
function pipe(...fns) {
  return function(value) {
    return fns.reduce((acc, fn) => fn(acc), value);
  };
}

const processSlug = pipe(trim, toLower, removeSpaces);
console.log(processSlug("  JavaScript Basics  "));  // "javascript-basics"
```

pipe and compose are the standard names for these composition utilities. pipe applies functions left to right (reads like a data pipeline), while compose applies them right to left (reads like nested function calls). pipe is generally more readable.

Example 3: Currying and Partial Application

```
// General multiply function
const multiply = a => b => a * b;

// Partial application — lock in the first argument
const double = multiply(2);
const triple = multiply(3);
const tenTimes = multiply(10);

console.log([1, 2, 3, 4, 5].map(double));    // [2, 4, 6, 8, 10]
console.log([1, 2, 3, 4, 5].map(triple));    // [3, 6, 9, 12, 15]
console.log([1, 2, 3, 4, 5].map(tenTimes));  // [10, 20, 30, 40, 50]

// Real-world currying: configurable validator
const isGreaterThan = min => value => value > min;

const isAdult = isGreaterThan(17);
const isOverFreezing = isGreaterThan(0);

console.log(isAdult(20));        // true
console.log(isOverFreezing(-5)); // false
```

```
const ages = [15, 18, 21, 12, 30];
console.log(ages.filter(isAdult));  // [18, 21, 30]
```

Example 4: HOFs for Error Handling

```
// A HOF that wraps any function with error handling
function withErrorHandling(fn) {
  return function(...args) {
    try {
      return fn(...args);
    } catch (error) {
      console.error(`Error in ${fn.name}: ${error.message}`);
      return null;
    }
  };
}

function parseJSON(str) {
  return JSON.parse(str);  // throws if invalid JSON
}

const safeParseJSON = withErrorHandling(parseJSON);

console.log(safeParseJSON('{"a": 1}'));  // { a: 1 }
console.log(safeParseJSON("invalid"));    // logs error, returns null
```

8.5 Edge Cases & Pitfalls

⚠Warning: Excessive currying or composition chains can harm readability. Use them when they genuinely simplify the code, not just to appear functional. When a simple for loop is clearer, use it.

```
// Callback Hell — too many nested callbacks
doA(function() {
  doB(function() {
    doC(function() {
      // deeply nested, hard to read
    });
  });
});

// Solution: named functions, or Promises/async-await (Chapter 10)
function afterA() { doB(afterB); }
function afterB() { doC(afterC); }
function afterC() { /* done */ }
```

```
doA(afterA);
```

Mini Practice Problems

1. Implement a myMap function from scratch, similar to Array.prototype.map. It should take an array and a transform function and return a new array.
2. Implement a myReduce function from scratch. Use it to find the maximum value in an array.
3. Write a pipe function that takes any number of functions and returns a new function that applies them left to right.
4. Write a once(fn) function (if you have not done so in Chapter 7) that returns a new function which can only be executed once. Subsequent calls return the first result.
5. Use currying to create a function that formats a price: formatPrice(currency)(amount) returns "$25.00" or "€25.00" depending on currency.

Chapter Summary

- A higher-order function accepts functions as arguments, returns functions, or both.
- Callbacks pass behavior as a parameter — the HOF provides the structure, the callback provides the logic.
- Function composition (pipe, compose) chains functions so the output of one becomes the input of the next.
- Currying transforms a multi-argument function into a chain of single-argument functions.
- Partial application pre-fills some arguments to create specialized versions of general functions.
- HOFs like map, filter, and reduce abstract the iteration pattern, letting you focus on the logic.
- Avoid callback hell by using named functions, Promises, or async/await.

Chapter 9

Logic Building

9.1 Concept Introduction

Logic building is the ability to break down a problem into a series of clear, executable steps. It is the translation of human intent into structured algorithmic thinking. Good logic building is not about knowing the most syntax or the most clever tricks — it is about approaching problems systematically, clearly, and with correctness in mind.

This chapter synthesizes everything learned so far and applies it to common programming patterns: sorting, searching, string manipulation, number theory, and algorithm design. The goal is to develop the thinking process, not just memorize solutions.

9.2 Core Theory

Problem Decomposition

Every complex problem can be broken into smaller, manageable sub-problems. Before writing a single line of code, ask: What is the input? What is the expected output? What are the steps to get from one to the other? Can any of those steps be broken down further? This is the foundation of algorithmic thinking.

Algorithmic Thinking

An algorithm is a finite set of clearly defined instructions for solving a problem. When designing one, consider: correctness (does it always produce the right answer?), edge cases (does it handle empty inputs, negative numbers, large values?), and efficiency (does it work within acceptable time and space?).

Thinking About Time Complexity (Informally)

Not all solutions are equal. A solution that loops through n items once is faster than one that loops n times for each of n items. As a beginner, focus on correctness first, then revisit efficiency. Ask: "Does my solution do unnecessary repeated work? Can I solve this in one pass instead of two?"

9.3 Common Patterns and Demonstrations

Pattern 1: Two-Pointer Technique

```
// Check if a string is a palindrome using two pointers
function isPalindrome(str) {
  let left = 0;
  let right = str.length - 1;

  while (left < right) {
    if (str[left] !== str[right]) return false;
    left++;
    right--;
  }
  return true;
}

console.log(isPalindrome("racecar"));  // true
console.log(isPalindrome("hello"));    // false
console.log(isPalindrome("a"));        // true (single character)
```

The two-pointer technique places one pointer at each end of a sequence and moves them toward each other. It avoids creating a reversed copy of the string and is more efficient. This pattern applies to sorted arrays, string problems, and search tasks.

Pattern 2: Frequency Counter

```
// Check if two strings are anagrams of each other
function areAnagrams(str1, str2) {
  if (str1.length !== str2.length) return false;

  const freq = {};

  // Count characters in str1
  for (let char of str1) {
    freq[char] = (freq[char] || 0) + 1;
  }

  // Subtract characters in str2
  for (let char of str2) {
    if (!freq[char]) return false;  // char not in str1, or count reached 0
    freq[char]--;
  }

  return true;
}

console.log(areAnagrams("listen", "silent"));  // true
console.log(areAnagrams("hello", "world"));    // false
```

The frequency counter pattern uses an object to record how often values appear. Instead of nested loops ($O(n^2)$), it solves many comparison problems in a single pass ($O(n)$) by tallying counts and then comparing them.

Pattern 3: Sliding Window

```
// Find the maximum sum of k consecutive elements
function maxSubarraySum(arr, k) {
  if (arr.length < k) return null;

  // Calculate sum of first window
  let windowSum = 0;
  for (let i = 0; i < k; i++) {
    windowSum += arr[i];
  }

  let maxSum = windowSum;

  // Slide the window: add next element, remove first element
  for (let i = k; i < arr.length; i++) {
    windowSum = windowSum + arr[i] - arr[i - k];
    maxSum = Math.max(maxSum, windowSum);
  }

  return maxSum;
}

console.log(maxSubarraySum([2, 6, 9, 2, 1, 8, 5, 6, 3], 3));  // 19
```

The sliding window technique maintains a "window" of fixed or variable size that moves through the data. Instead of re-summing all k elements for each position (which would be O(n*k)), we efficiently update the sum by subtracting the element that left the window and adding the new one (O(n)).

Pattern 4: Recursion for Tree-Like Problems

```
// Flatten deeply nested arrays recursively
function flatten(arr) {
  let result = [];

  for (let item of arr) {
    if (Array.isArray(item)) {
      result = result.concat(flatten(item));  // recursive call
    } else {
      result.push(item);
    }
  }

  return result;
}

console.log(flatten([1, [2, [3, [4]], 5]]));  // [1, 2, 3, 4, 5]
```

Recursion is natural for tree-like or nested structures because the sub-problems have the same shape as the original problem. An array can contain arrays which contain arrays — recursion handles each level uniformly.

Pattern 5: Building Solutions Incrementally

```
// FizzBuzz — classic logic building exercise
// For numbers 1-100: print Fizz for multiples of 3,
// Buzz for multiples of 5, FizzBuzz for multiples of both
function fizzBuzz(n) {
  for (let i = 1; i <= n; i++) {
    if (i % 15 === 0) console.log("FizzBuzz"); // check BOTH first
    else if (i % 3 === 0) console.log("Fizz");
    else if (i % 5 === 0) console.log("Buzz");
    else console.log(i);
  }
}

// A more scalable approach using string building
function fizzBuzz2(n) {
  for (let i = 1; i <= n; i++) {
    let output = "";
    if (i % 3 === 0) output += "Fizz";
    if (i % 5 === 0) output += "Buzz";
    console.log(output || i);
  }
}
```

The second approach is more scalable. If a new rule is added (e.g., multiples of 7 print "Jazz"), you only add one line. The first approach would require adding many new conditions. Always consider how your solution handles change.

Pattern 6: Prime Numbers

```
function isPrime(n) {
  if (n < 2) return false;
  if (n === 2) return true;
  if (n % 2 === 0) return false;  // optimization: skip even numbers

  // Only check up to the square root — optimization
  // If n has a factor > sqrt(n), the other factor must be < sqrt(n)
  for (let i = 3; i <= Math.sqrt(n); i += 2) {
    if (n % i === 0) return false;
  }
  return true;
}

console.log(isPrime(7));   // true
console.log(isPrime(12));  // false
console.log(isPrime(97));  // true
```

```
// Get all primes up to n (Sieve-inspired)
function primesUpTo(n) {
  return Array.from({ length: n }, (_, i) => i + 1)
    .filter(isPrime);
}
console.log(primesUpTo(30));  // [2, 3, 5, 7, 11, 13, 17, 19, 23, 29]
```

9.4 Edge Cases & Pitfalls

```
// Edge cases to always consider

// 1. Empty input
function sumArray(arr) {
  if (!arr || arr.length === 0) return 0;  // handle empty
  return arr.reduce((s, n) => s + n, 0);
}

// 2. Negative numbers and zero
function factorial(n) {
  if (n < 0) throw new Error("Negative input not allowed");
  if (n === 0) return 1;  // 0! = 1 by definition
  return n * factorial(n - 1);
}

// 3. Type checking
function double(n) {
  if (typeof n !== "number") throw new TypeError("Expected a number");
  return n * 2;
}
```

➖Mini Practice Problems

1. Write a function that finds the two numbers in an array that sum to a given target. Return their indices. Example: twoSum([2, 7, 11, 15], 9) → [0, 1].
2. Write a function that counts the number of vowels in a string. Then extend it to count each vowel separately.
3. Given a string, write a function to find the first character that does not repeat. Example: "swiss" → "w".
4. Write a function that converts a number to Roman numerals. For example: 2024 → "MMXXIV".
5. Implement a binary search function: given a sorted array and a target value, return the index or -1 if not found. Explain why binary search is faster than linear search.

✓Chapter Summary

- Logic building is the process of translating a problem into step-by-step algorithmic instructions.
- Two-pointer technique: use two indices moving toward each other — useful for palindromes, sorted arrays.
- Frequency counter: use an object to count occurrences — avoids nested loops in comparison problems.
- Sliding window: maintain a moving subset of data — ideal for contiguous subarray problems.
- Recursion is natural for nested or tree-like problems — always define a base case.
- Always handle edge cases: empty inputs, zero, negative numbers, and type errors.
- Correctness first, efficiency second. Once working, look for unnecessary repeated work to optimize.

Chapter 10

Advanced JavaScript

10.1 Concept Introduction

This chapter covers the advanced mechanisms that separate intermediate JavaScript developers from advanced ones: Promises and asynchronous programming, the class system and prototypal inheritance, Iterators and Generators, and modern ES features. These concepts build on everything covered in the previous chapters.

10.2 Asynchronous JavaScript

The Event Loop

JavaScript executes in a single thread, processing tasks one at a time from the Call Stack. However, operations like network requests or timers can take time. If JavaScript blocked the entire thread while waiting, the UI would freeze. To handle this, JavaScript uses an event-driven concurrency model powered by the Event Loop.

When an asynchronous operation is initiated (e.g., setTimeout, fetch), it is handed off to the browser or Node.js runtime. JavaScript continues executing other code. When the async operation completes, its callback is placed in the Callback Queue (or Microtask Queue for Promises). The Event Loop constantly checks: is the Call Stack empty? If yes, it picks the next task from the queue and pushes it onto the stack.

> **Microtask Queue vs Callback Queue**
> Promise callbacks (.then, .catch) go into the Microtask Queue, which is processed before the Callback Queue (where setTimeout callbacks go). This means Promise resolutions always happen before setTimeout callbacks, even if the timeout is 0ms.

Promises

A Promise is an object that represents the eventual completion or failure of an asynchronous operation. It has three states: pending (the operation has not completed yet), fulfilled (the operation succeeded, and the promise has a result value), and rejected (the operation failed, and the promise has an error reason). Once a promise transitions to fulfilled or rejected, it is settled and its state never changes.

```
// Creating a Promise
const fetchUser = (id) => {
  return new Promise((resolve, reject) => {
    // Simulate async operation
```

```
    setTimeout(() => {
      if (id > 0) {
        resolve({ id, name: "Alice" });  // success
      } else {
        reject(new Error("Invalid user ID"));  // failure
      }
    }, 1000);
  });
};

// Consuming a Promise
fetchUser(1)
  .then(user => console.log("Got user:", user.name))
  .catch(err => console.error("Error:", err.message))
  .finally(() => console.log("Request finished."));

// Promise.all — wait for multiple promises
Promise.all([fetchUser(1), fetchUser(2)])
  .then(([user1, user2]) => console.log(user1, user2))
  .catch(err => console.error("One failed:", err));
```

async and await

async/await is syntactic sugar built on top of Promises. An async function always returns a Promise. Inside an async function, the await keyword pauses execution until a Promise settles, then returns its resolved value. This makes asynchronous code look and behave like synchronous code, dramatically improving readability.

```
// async/await version — reads like synchronous code
async function loadUserData(id) {
  try {
    const user = await fetchUser(id);          // waits for promise
    console.log("User:", user.name);

    const posts = await fetchPosts(user.id);   // runs after user resolves
    console.log("Posts:", posts.length);

    return { user, posts };
  } catch (error) {
    console.error("Failed:", error.message);
  }
}

loadUserData(1);

// Running multiple in parallel (do NOT await sequentially if independent)
async function loadParallel() {
  const [user, posts] = await Promise.all([fetchUser(1), fetchPosts(1)]);
  return { user, posts };
}
```

A common mistake with async/await is awaiting promises sequentially when they could run in parallel. If fetchUser and fetchPosts do not depend on each other, use Promise.all so they run simultaneously and the total wait time is the maximum of the two, not their sum.

10.3 Classes and Prototypal Inheritance

JavaScript's class syntax (introduced in ES6) is a cleaner way to create objects that share behavior through prototypes. Classes are not a new inheritance model — they are syntactic sugar over the prototype chain that existed in JavaScript from the beginning.

```
class Animal {
  constructor(name, sound) {
    this.name = name;
    this.sound = sound;
  }

  speak() {
    return `${this.name} says ${this.sound}!`;
  }

  toString() {
    return `[Animal: ${this.name}]`;
  }
}

// Inheritance with extends
class Dog extends Animal {
  constructor(name) {
    super(name, "woof");  // must call super() before using this
    this.tricks = [];
  }

  learn(trick) {
    this.tricks.push(trick);
    return this;
  }

  perform() {
    return this.tricks.length > 0
      ? `${this.name} performs: ${this.tricks.join(", ")}`
      : `${this.name} knows no tricks yet.`;
  }
}

const rex = new Dog("Rex");
rex.learn("sit").learn("shake").learn("roll over");
console.log(rex.speak());    // "Rex says woof!"
console.log(rex.perform());  // "Rex performs: sit, shake, roll over"
console.log(rex instanceof Dog);    // true
```

```
console.log(rex instanceof Animal); // true
```

When rex.speak() is called, JavaScript first checks rex itself for a speak property. It is not there (only name, sound, and tricks are own properties). JavaScript then checks Dog.prototype — not there either. Then it checks Animal.prototype — there it is! This traversal is the prototype chain in action.

Static Methods and Private Fields

```
class BankAccount {
  #balance = 0;  // private field (ES2022) — cannot be accessed outside
  static interestRate = 0.05;  // static — belongs to the class, not
instances

  constructor(owner, initialDeposit) {
    this.owner = owner;
    this.#balance = initialDeposit;
  }

  deposit(amount) {
    this.#balance += amount;
    return this;
  }

  get balance() {
    return this.#balance;  // getter — accessed like a property
  }

  static create(owner, deposit) {
    return new BankAccount(owner, deposit);
  }
}

const acc = BankAccount.create("Alice", 1000);
acc.deposit(500);
console.log(acc.balance);   // 1500
// console.log(acc.#balance); // SyntaxError — private!
console.log(BankAccount.interestRate);  // 0.05
```

10.4 Iterators and Generators

An iterator is any object that implements a next() method which returns {value, done}. When done is true, the iteration is complete. The for...of loop and the spread operator both work with iterables — objects that have a [Symbol.iterator] method that returns an iterator.

A generator function (declared with function*) automatically creates an iterator. The yield keyword pauses the generator and produces a value. The next time next() is called, execution

resumes from where it left off. Generators are ideal for lazy sequences, infinite data streams, and custom iteration logic.

```
// Generator function
function* range(start, end, step = 1) {
  for (let i = start; i <= end; i += step) {
    yield i;  // pauses here, produces i
  }
}

// Use with for...of
for (let num of range(1, 10, 2)) {
  console.log(num);  // 1, 3, 5, 7, 9
}

// Infinite generator — generates values on demand
function* fibonacci() {
  let [a, b] = [0, 1];
  while (true) {
    yield a;
    [a, b] = [b, a + b];
  }
}

const fib = fibonacci();
console.log(fib.next().value);  // 0
console.log(fib.next().value);  // 1
console.log(fib.next().value);  // 1
console.log(fib.next().value);  // 2
console.log(fib.next().value);  // 3
// Values are generated one at a time — no infinite loop!
```

10.5 Modern JavaScript Features

```
// Destructuring (revisited in depth)
const [first, second, ...rest] = [1, 2, 3, 4, 5];
console.log(first);  // 1
console.log(rest);   // [3, 4, 5]

// Object shorthand and computed properties
const field = "name";
const value = "Alice";
const obj = { [field]: value };  // { name: "Alice" }

// Nullish Assignment (??=)
let config = { debug: null };
config.debug ??= false;  // only assigns if null or undefined
console.log(config.debug);  // false

// Optional chaining with method calls
```

```
const user = null;
user?.getName?.();  // undefined — no error thrown

// Promise.allSettled — does not reject if one fails
const results = await Promise.allSettled([
  fetchUser(1),
  fetchUser(-1),  // this rejects
  fetchUser(2)
]);
// results: [{status:"fulfilled", value:...}, {status:"rejected",
reason:...}, ...]
```

10.6 Edge Cases & Pitfalls

⚠Warning: Forgetting to await a Promise means you are working with the Promise object itself, not its resolved value. Always await inside async functions, or use .then() for promise chains.

```
// Common async mistake
async function getUser() {
  const user = fetchUser(1);  // forgot await!
  console.log(user.name);     // undefined — user is a Promise, not the data

  const userFixed = await fetchUser(1);  // correct
  console.log(userFixed.name);           // "Alice"
}

// Unhandled promise rejection
async function riskyOp() {
  throw new Error("Something broke");
}

// Without .catch or try/catch, this is an unhandled rejection
riskyOp();  // silent failure in some environments!

// Always handle errors
riskyOp().catch(err => console.error(err));
// or
try { await riskyOp(); } catch(e) { console.error(e); }
```

➖Mini Practice Problems

1. Write an async function that fetches data from two endpoints in parallel and returns a combined result. Use Promise.all.

2. Create a class Shape with a constructor for color, and two subclasses: Circle (with radius) and Rectangle (with width and height). Each should have an area() method.

3. Write a generator function that produces prime numbers indefinitely. Use it to get the first 10 primes.
4. Explain the difference between Promise.all and Promise.allSettled. Give a scenario where you would use each.
5. What is the Event Loop? Predict the output order of: console.log("A"); setTimeout(() => console.log("B"), 0); Promise.resolve().then(() => console.log("C")); console.log("D");

✓Chapter Summary

- JavaScript is single-threaded; asynchronous operations are handled by the Event Loop.
- Promises represent eventual success or failure of async operations. Chains: .then, .catch, .finally.
- async/await makes async code readable; always use try/catch for error handling.
- Microtask Queue (Promises) is processed before the Callback Queue (setTimeout).
- Classes are syntactic sugar over prototypes. extends and super enable inheritance.
- Private fields (#field) provide true encapsulation. Static members belong to the class itself.
- Generators produce values lazily using yield — ideal for sequences and streams.
- Promise.all fails fast if any reject; Promise.allSettled waits for all regardless of outcome.